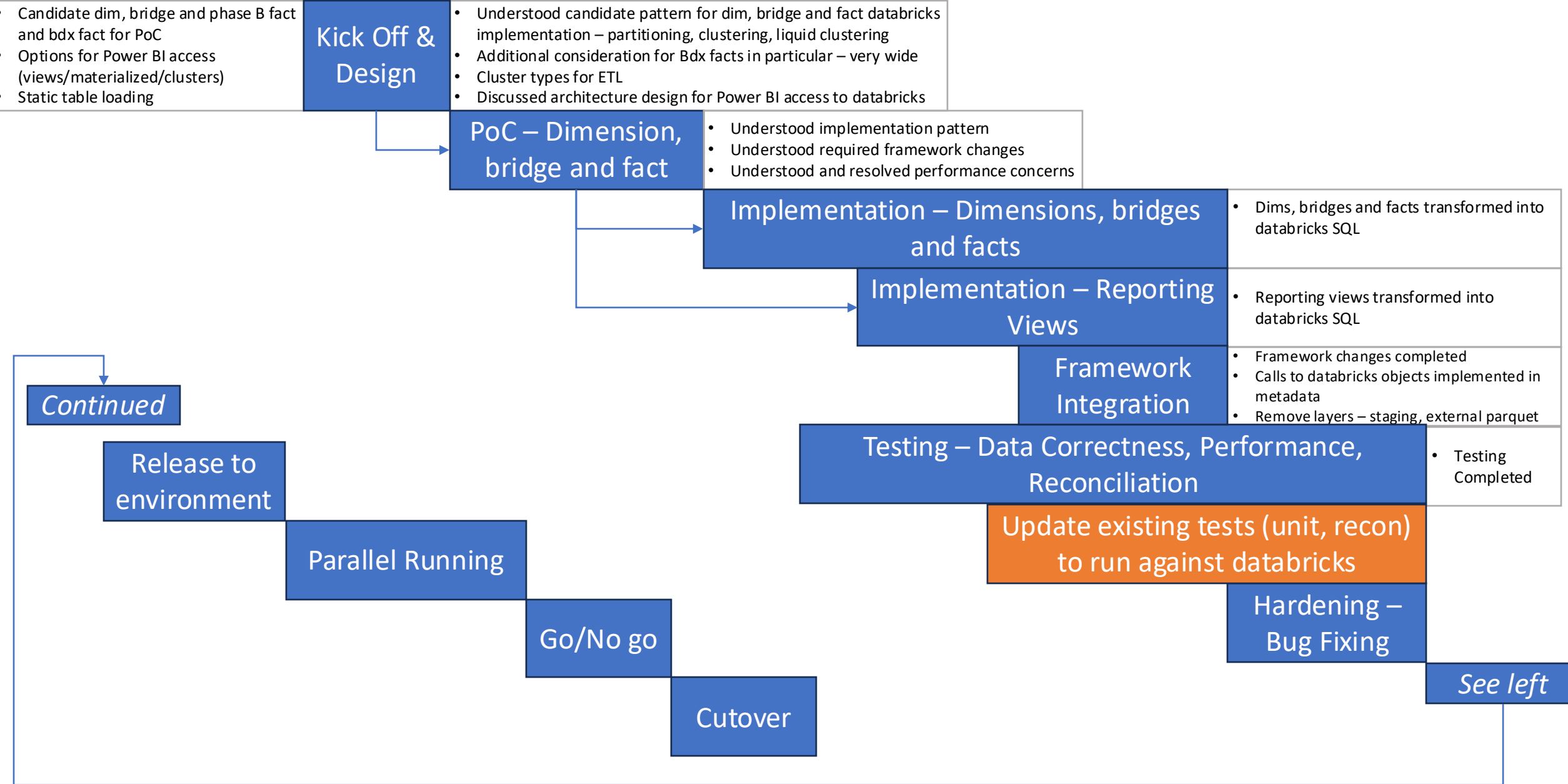


Synapse -> Databricks Migration

VERY ROUGH DRAFT



Items to be discussed internally

- Identify SQL Functions -HBKs, convertmonthyeartodateid, convertmonthyeartomonthid--check usage of least of 2 dates, space before cap functions in synapse, replicate in databricks --Sourav
- Case Sensitive consideration - used upper in joins, where condition on text cols, should we apply trim? discussion on applying upper(trim(cols)) moving from raw to std
- Addition of source ID's in master view - should come through cnf, update existing hbks
- Generic code for defaults - this should be a single notebook that goes through all tables adding defaults for certain types where appropriate
- Identify Generation - generic notebook to insert default values in dims, discussion to replace null to sensible values in dims
- Progress on alternative fact structure for loading -- create a flowchart

table creation and data migration from synapse to databricks, hbk updates (Sourav)

- Extracted Tables,Columns metadata from Synapse and stored in Dataframes.
- Updated the datatype mapping between source and target, Generated DDL statements for Databricks SQL and save them as separate files in migration storage account.
- Created Delta tables by executing the same generated DDL statements in Databricks SQL.
- To export data, Created an external table for every user table. This actually exports data to staging storage.
- Import data into lakehouse from staging storage using COPY INTO statements. Data is converted to Delta format at this point.

Dim Outwards Policy - nikunj

- Build on top of conformed masterviews (used staging in synapse)
- HBKs generated using the same function that generates HBKs in conformed layer (stored as bigint as compared to varbinary in synapse)
- Code written in SQL commands
- Created temp views as compared to CTAS in synapse
- Overall structure of the code still remains the same
- Clustered on valid from and to timestamps

Dim BDX - KK

- Build on top of conformed masterviews (used staging in synapse)
- HBKs generated using the same function that generates HBKs in conformed layer (stored as bigint as compared to varbinary in synapse)
- Code written in SQL commands
- Created temp views as compared to CTAS in synapse
- Updates and inserts are combined to a single merge statement
- Try/catch and roll back to initial version as part of the first merge
- Second merge statement for reactivated records
- Third merge for updating original_id

Bridge Line Outwards Policy - Nikunj

- Build on top of conformed masterviews (used staging in synapse) and copied dims (from synapse)
- HBKs in bridge generated using the same function that generates HBKs in conformed layer (stored as bigint as compared to varbinary in synapse)
- HBKs were updated in the copied dims (so that the joins could happen)
- Created temp views as compared to CTAS in synapse
- Code written in SQL commands
- Overall structure of the code still remains the same
- Clustered on valid from and to timestamps

Fact outwards settlement v1- Nikunj

- Build on top of conformed masterviews (used staging in synapse) and copied dims (from synapse)
- HBKs in fact generated using the same function that generates HBKs in conformed layer (stored as bigint as compared to varbinary in synapse)
- HBKs were updated in the copied dims (so that the joins could happen)
- Code written in SQL commands
- Overall structure of the code was the same, the load took around 3 mins 20 sec
- Clustered on valid from and to timestamps

Fact outwards settlement v2- Nikunj

- This approach had 2 changes;
 - Rather than joining again and again on the base table and bringing in all the data in each join, separate base tables were created, joined with the required dims, and all these base table joined at once in the final step
 - Accounting and development period updates, that used to happen post the fact loads in synapse, was updated so that it gets handled in a single insert
- This provided better timings, v1 took 3 mins 20 sec whereas v2 took around 1 min 50 sec for the complete load

Fact BDX - KK

Summary Timings KK/Nikunj

Entity	Time Taken in Synapse	Time Taken in Databricks
Dim_Outwards_Policy	24 sec	23 sec
Dim_Bdx		22 sec
Bridge_Line_Outwards_Policy	25 sec	25 sec
Fact_Outwards_Settlement	6 mins 19 sec	1 min 50 sec
Fact_Bdx_		1 hour

Summary Timings KK/Nikunj

Entity	Time Taken in Synapse	Time Taken in Databricks	Cost Synapse	Cost Databricks
Dim_Outwards_Policy	0.4	0.4		
Dim_Bdx		0.36		
Bridge_Line_Outwards_Policy	0.42	0.42		
Fact_Outwards_Settlement	3.33	1.8		
Fact_Bdx_		60		

Cost Comparison DB vs Synapse - Chris

Concers/Queries - Chris/KK/Sourav/Nikunj

- HBK, HVL updates across Dims, Bridges and Facts post data copy from synapse
- IDs and Original ID updates post data copy from synapse
- Division by 0 error in databricks

Concerns / Unknowns	Answer	Resolved
Cluster Type – SQL Warehouse? Or do we NEED python – need to think about logging to metadata DB	Python clusters	Y
If SQL Warehouse then how do we log to the SQL database – or do we just log to a delta table (high concurrency...)	NA	Y
Limit of concurrent connections to databricks sql endpoint, 10? (power bi refresh impact) – Distribute tables across end points?	This would be controlled by the size of the cluster and the scalability of the number of clusters	Y
Strategy for temp tables. We use a lot during our load of facts (in particular) in synapse dedicated pool. Should we follow the same approach?	https://www.databricks.com/discover/pages/optimize-data-workloads-guide#intermediate-results	
Lots of code we have (and could take advantage of) is in python functions. Moving this over to SQL Functions could be tricky...	Identify the functions that are used frequently inline in the SQL code and understand can be moved to SQL Functions. The exact items are still TBD	Y
Identity generation?	HBK code can be translated fairly easily. Identity generation is - GENERATED BY DEFAULT AS IDENTITY	N
Re-run of DWH entities- currently there is a different log table in synapse that prevents re-running of proc for same BED, once we bring dwh to databricks, whether we'll integrate the synapse and databricks log tables or dwh logging will be different from conformed	Canopus/EXL decision	
Decision – clean and load or migrate data – Keys and ID's HBK, HV and ID should get updated if we are not planning for a truncate load	Canopus/EXL decision	
Approach for incremental load should be based on where clause or tbl_bed_history logic in dwh tables	Canopus/EXL decision	

Known changes		Resolved
Case sensitivity along with leading/trailing blanks in databricks (not in synapse)	Canopus/EXL	
Functions changes -> produce a mapping table with confirmed values being the same, e.g. getdate() ->		
Data type changes	These are documented in Synapse%20Migration%20Accelerator%20overview.pdf	Y
Delta table creation – metadata for these tables need to be created in metadata so notebook can create them – not done in synapse currently	Canopus/EXL for the metadata – also assisted by the synapse-migration-accelerator.dbc notebook and Synapse%20Migration%20Accelerator%20overview.pdf	Y
Handling of BED currently up to staging needs to be reimplemented	Canopus/EXL	
MasterViews, External Parquet, External Staging and Staging layers can be merged as one	Canopus/EXL	
Add source ID as part of HBK in conformed	Canopus/EXL	

Next Steps

Action Items	Queries
Is it the logic or the volume that dramatically increases the run time?	- Replicate the transactions in the 3 source transaction tables. These will be also added to the dimension so lookups can work. - We will look to increase the volume by 10 from ~3 million to ~30 million
How do the cost change while maintaining the performance when more work is happening in parallel.	- Using the revised volume of the fact at ~30 million rows – create 10 versions of the transaction source table, target table and the notebooks. - Run the variants in parallel.

Source View	Row Count	Target Table for testing	Row Count
insights_dev_001.conformedmasterviews.vwoutwards_settlement_detail	5,938,034	insights_dev_002.poc_synapse_databricks.conformedmasterviews_vwoutwards_settlement_detail	59,380,340
insights_dev_001.conformedmasterviews.vwoutwards_transaction_link	805,426	insights_dev_002.poc_synapse_databricks.conformedmasterviews_vwoutwards_transaction_link	8,054,260
insights_dev_001.conformedmasterviews.vwoutwards_transaction	1,178,200	insights_dev_002.poc_synapse_databricks.conformedmasterviews_vwoutwards_transaction	1,1782,000